

WIARYGODNOŚĆ

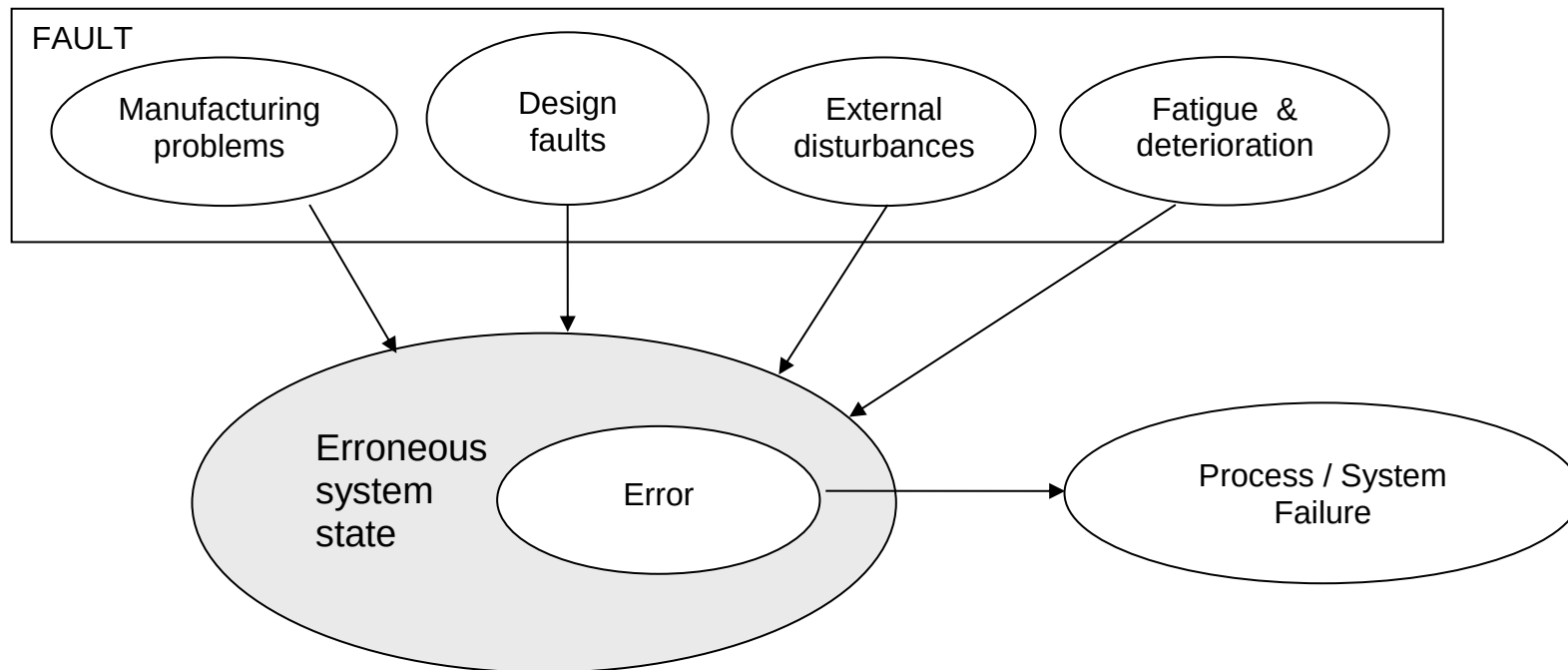
= pewność działania systemu, która pozwala mieć uzasadnione zaufanie do usług, które ten system dostarcza

System wiarygodny [1]

- = niezawodny (*reliable*) – odporny na awarie
- = dyspozycyjny (*available*) – dostępny na bieżąco
- = bezpieczny (*secure*) – zapewniający ochronę danych
- = bezpieczny (*safe*) – dla środowiska

Podstawowe definicje [2]

- **awaria** (*failure*) – występuje gdy system przestaje być zgodny ze specyfikacją
- **błąd** (*error*) – część stanu systemu odpowiedzialna za będącą jego następstwem awarię – symptom defektu, który właśnie lub wcześniej wystąpił
- **defekt/uszkodzenie** (*fault*) – stwierdzona lub hipotetyczna przyczyna wystąpienia błędu



NARUSZENIA NIEZAWODNOŚCI

Defekty/uszkodzenia

Źródło uszkodzenia:

- uszkodzenia fizyczne
- uszkodzenia spowodowane przez człowieka

Granice źródła uszkodzenia:

- uszkodzenia wewnętrzne
- uszkodzenia zewnętrzne

Faza powstania uszkodzeń:

- uszkodzenia projektowe
- uszkodzenia operacyjne

Trwałość uszkodzeń:

- uszkodzenia trwałe (*permanent*)
- uszkodzenia przemijające (*transient*) / przerywane (*intermittent*)

Przykład 1:

- uszkodzenie projektowe jest wynikiem awarii spowodowanej przez projektanta

Swap $A[i] \leftrightarrow A[j]$

```
swap(in i, j)
```

```
{
```

```
    A[i] := A[i] - A[j];
```

```
    A[j] := A[j] + A[i];
```

```
    A[i] := A[j] - A[i];
```

```
}
```

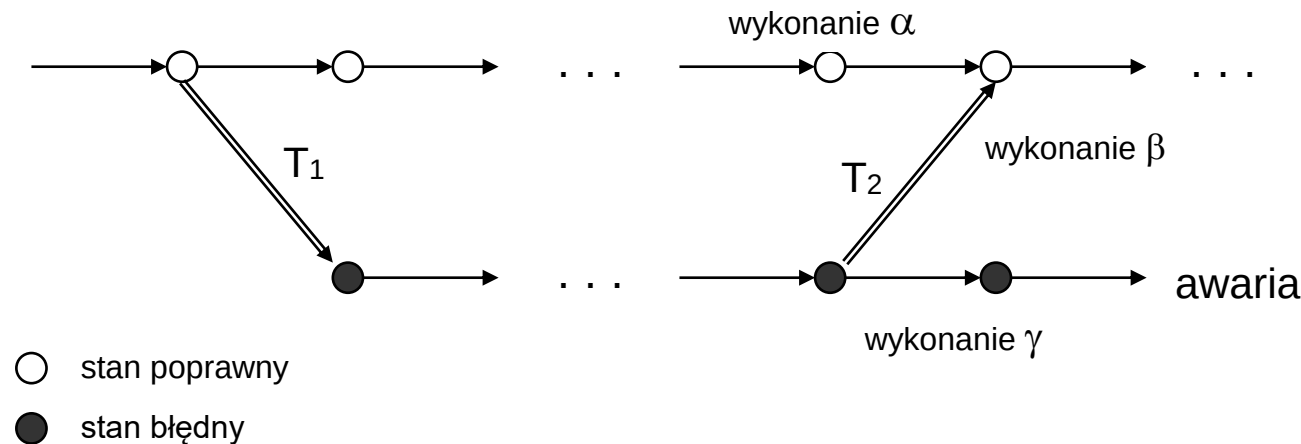


Przykład 2:

- uszkodzenie zewnętrzne fizyczne lub spowodowane przez człowieka jest *de facto* uszkodzeniem projektowym wynikającym z niemożności przewidzenia wszystkich sytuacji lub rezygnacji z rozpatrywania niektórych z nich (np. wystąpienie zakłócenia elektromagnetycznego: czy jest to uszkodzenie zewnętrzne, czy brak odpowiedniego ekranowania?)

Błędy

- błąd odpowiada za awarię
- wystąpienie błędu doprowadzi faktycznie do awarii w zależności od:
 - budowy systemu (w szczególności np. istnienia redundancji)
 - działania systemu (błąd może zostać skasowany np. przez zapisanie nowych danych zanim spowoduje szkody)
 - postrzegania przez użytkownika (np. czas przestoju systemu może być, albo nie, postrzegany jako awaria w zależności od ziarnistości czasu konkretnego użytkownika)



Awarie

Kryterium dziedziny awarii:

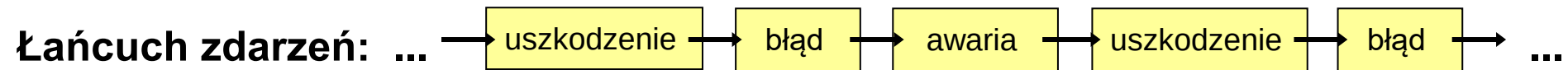
- wynikające z błędów wartości (*value failures*)
- wynikające z błędów synchronizacji (*timing failures*)

Kryterium percepcji awarii:

- awarie zgodne (*consistent failures*) – wszyscy użytkownicy zgodnie postrzegają tę samą awarię
- awarie niezgodne (*inconsistent failures*) – użytkownicy różnie postrzegają tę samą awarię

Kryterium dotkliwości awarii:

- awarie łagodne (*benign failures*) – konsekwencje są tego samego rzędu, co korzyści wynikające z dostarczanej usługi przy braku awarii
- awarie katastroficzne (*catastrophic failures*) – konsekwencje nieporównywalnie większe





Podstawowe klasy awarii:

- **zatrzymania** – praca systemu (lub jego komponentu) przestaje być dostrzegalna dla użytkowników – wynikają zarówno błędów wartości jak i synchronizacji
 - **zaniechanie** (*omission failures*) – specjalny typ klasy zatrzymania: zatrzymanie chwilowe (np. zaniechanie dostarczenia wiadomości przez kanał komunikacyjny)
 - graniczny przypadek konsekwencji błędów wartości (wartość nieistotna) i synchronizacji (nieskończone opóźnienie)
 - **upadek** (*crash failures*) – trwała awaria zatrzymania
- **bizantyjskie** (*Byzantine failures*, *malicious failures*) – arbitralne awarie niezgodne (najogólniejsza klasa)

Modele awarii

- ***fail-silent*** – awarie są klasy zatrzymanie (np. zaniechanie) i nie muszą być zauważalne
- ***fail-stop*** – awarie klasy upadek (*crash*) i są zauważalne
- ***fail-recovery*** – awarie klasy upadek i uszkodzone elementy są reaktywowane po zatrzymaniu
- ***Byzantine*** – awarie w systemie mogą być typu bizantyjskiego

MODELE SYSTEMU

Środowisko rozproszone

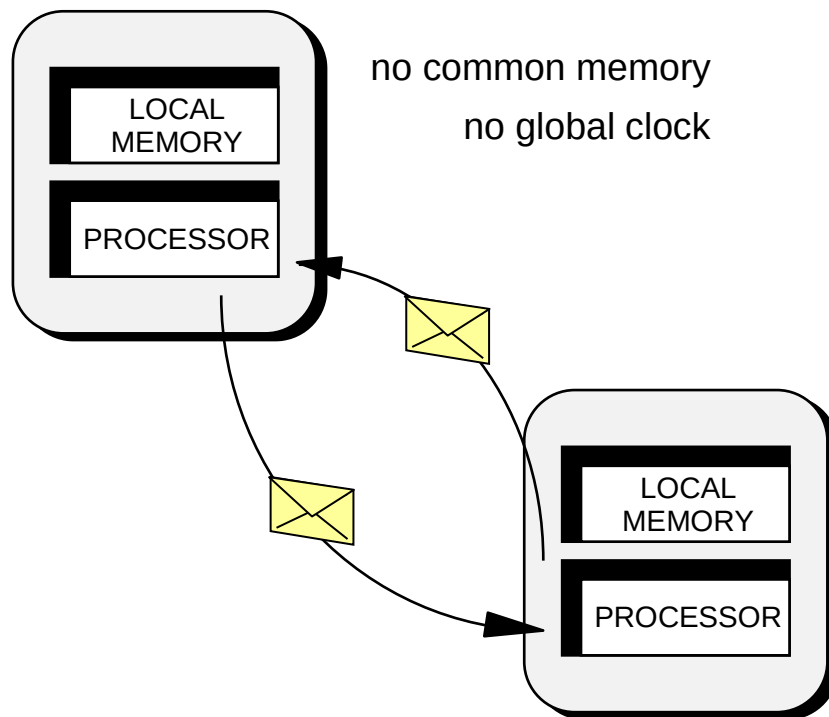
Niedeterminizm przetwarzania:

- nieznana liczba procesów
- nieznana topologia sieci
- współbieżność procesów uruchamianych w różnych momentach czasu i wykonywanych z różnymi prędkościami
- niezależne dane wejściowe
- indeterminizm procesorów
- nieokreślone opóźnienia komunikacyjne
- nieokreślone uporządkowanie wiadomości
- zawodność procesów i łączności komunikacyjnych
- heterogeniczność elementów

Modele komunikacji

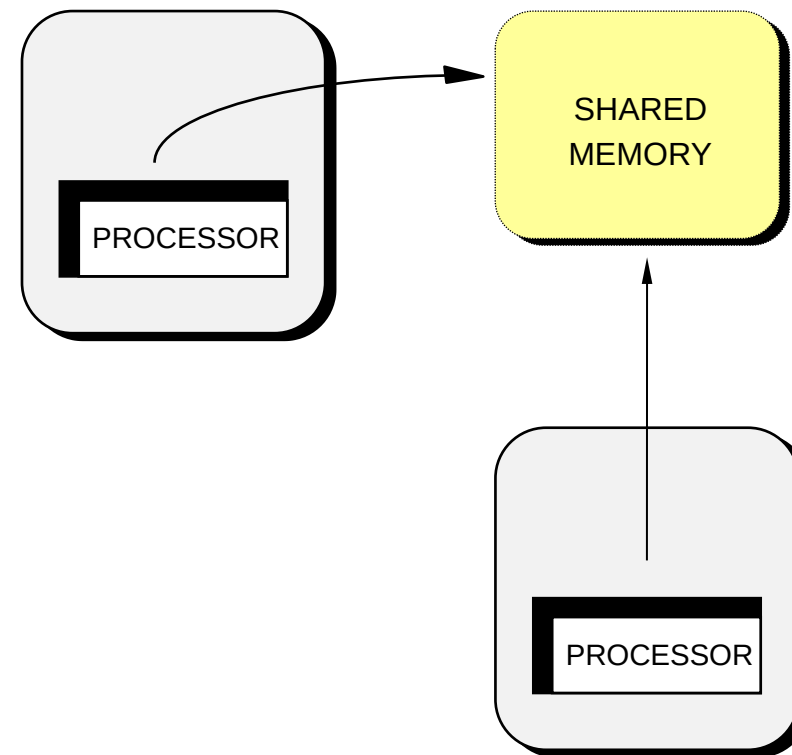
Message Passing

easy to implement
hard to program



Distributed Shared Memory

hard to implement
easy to program



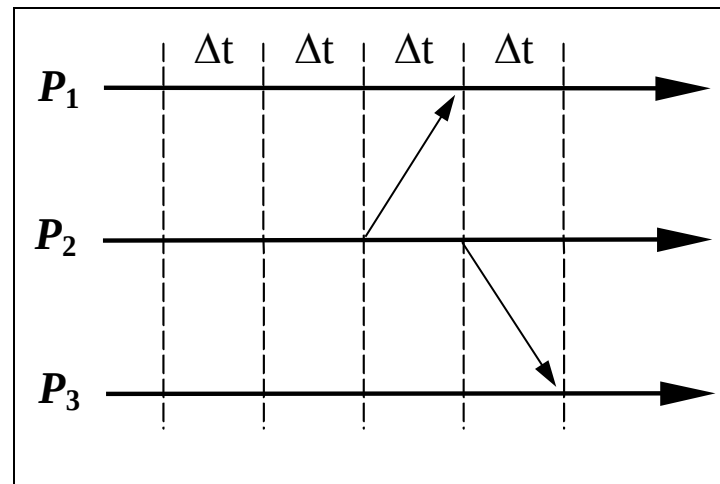
Graf połączeń G (*communication graph*)

- $distance(i,j)$ = min. ścieżka między P_i a P_j w G
- $diameter(G) = \max_{i,j} distance(i,j)$

Modele przetwarzania

1) system synchroniczny $\mathbb{S}^{\text{Sync}}$

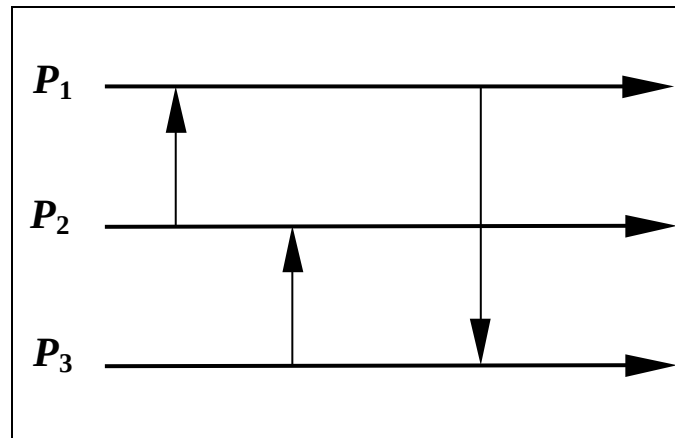
- kroki algorytmu wykonywane jednocześnie przez wszystkie procesy (*synchronous rounds*)



- znane ograniczenia:
 - 🕒 prędkości procesorów
 - ✈️ opóźnień komunikacyjnych
- możliwa synchronizacja zegarów

przykłady synchronicznej komunikacji

☞ komunikacja natychmiastowa:



☞ jednostkowe czasy komunikacji

☞ komunikacja poprzez współdzielone zmienne:

- *state-reading model*
- *link-register model*

2) system asynchroniczny $\mathbb{S}^{\text{Async}}$

- niezależne wykonywanie operacji przez procesy, w różnym tempie, skończonym, ale nieznanym
- opóźnienia komunikacyjne skończone, lecz nieznanne

3) system quasi-synchroniczny (*timing-based*)

- ograniczenia względnych prędkości procesów i czasów komunikacji istnieją, lecz nie są znane
- ograniczenia są znane, lecz występują dopiero po nieznanym z góry czasie
- ograniczenia nie są znane i występują dopiero po nieznanym czasie
- ograniczenia występują tylko w czasie pewnych nieznanym z góry okresów (*stable periods*)
- ... i 30 innych modeli

Przetwarzanie sterowane zdarzeniami (*event-driven*)

Procesy:

- uporządkowana sekwencja niepodzielnych zdarzeń
- czas trwania zdarzenia – przyjmujemy natychmiastowość zdarzeń
- stan s_i^k procesu P_i jest zbiorem wartości jego zmiennych lokalnych

Ogólne typy zdarzeń:

- $init(P_i, s_i^k)$ – lokalny stan procesu P_i zostaje zainicjowany wartością s_i^k ;
wyróżniamy wirtualne zdarzenie $e^0 = init^0(P_i, s_i^0)$ rozpoczynające przetwarzanie procesu P_i
- $send(P_i, P, m)$ – P_i wysyła wiadomość m do procesów $P_j \in P$
- $recv(P_i, P_j, m)$ – P_i odbiera wiadomość m od procesu P_j
- $internal(P_i)$ – P_i zmienia wartości zmiennych lokalnych
- $stop(P_i)$ – P_i kończy działanie
- $fail(P_i)$ – występuje awaria w procesie P_i

Stan lokalny procesu P_i :

- w chwili t jest reprezentowany sekwencją zdarzeń $e_i^0, e_i^1, \dots, e_i^t$, (zwaną historią), które zaszły od momentu wystąpienia zdarzenia $init^0(P_i, s_i^0)$ do chwili t
- zdarzenia zachodzące na procesie P_i są w pełni uporządkowane
- zdarzenia w całym przetwarzaniu rozproszonym są częściowo uporządkowane relacją poprzedzania $\rightarrow$ (*happen before*) [3] określającą porządek przyczynowy (*causal precedence*)
- stan będący efektem zajścia zdarzenia e_i^k oznaczamy s_i^k
- stany procesów są również częściowo uporządkowane następującą relacją poprzedzania $\rightarrow$:

$$s_i^k \rightarrow s_j^l \Leftrightarrow e_i^{k+1} \rightarrow e_j^l \text{ lub } e_i^{k+1} = e_j^l$$

Stan globalny systemu rozproszonego

- jest wektorem stanów lokalnych procesów i stanów kanałów komunikacyjnych

$$\langle s_1^t, s_2^t, \dots, s_n^t \rangle \cup \langle \dots, c_{ij}^t, \dots \rangle$$

- obraz stanu globalnego $\Sigma = \langle s_1^{t_1}, s_2^{t_2}, \dots, s_n^{t_n} \rangle \cup \langle \dots, c_{ij}^{t_c}, \dots \rangle$

- obraz stanu globalnego jest spójny $\Leftrightarrow$

$$1) (e \in \Sigma \wedge e' \rightarrow e) \Rightarrow e' \in \Sigma$$

$$2) e = \text{send}(\mathbf{P}_i, \mathbf{P}_j, m) \wedge e' = \text{recv}(\mathbf{P}_j, \mathbf{P}_i, m): \\ (e \in \Sigma \wedge e' \in \Sigma) \Rightarrow m \notin \Sigma$$

$$3) e = \text{send}(\mathbf{P}_i, \mathbf{P}_j, m) \wedge e' = \text{recv}(\mathbf{P}_j, \mathbf{P}_i, m): \\ (e \in \Sigma \wedge e' \notin \Sigma) \Rightarrow m \in \Sigma$$

$$4) e = \text{send}(\mathbf{P}_i, \mathbf{P}_j, m): e \notin \Sigma \Rightarrow m \notin \Sigma$$

- obraz stanu globalnego jest silnie spójny $\Leftrightarrow$ jest spójny oraz

$$\forall_{e, e'} e = \text{send}(\mathbf{P}_i, \mathbf{P}_j, m) \wedge e' = \text{recv}(\mathbf{P}_j, \mathbf{P}_i, m): e \in \Sigma \Leftrightarrow e' \in \Sigma$$

Poprawność programu rozproszonego

1. Własność **bezpieczeństwa** (ang. *safety*, *consistency*)

Program rozproszony jest bezpieczny, jeśli nigdy nie dopuszcza do niepożądanego stanu lub inaczej zawsze utrzymuje system w stanie pożądanym.

2. Własność **żywotności** / **postępu** (ang. *liveness*, *progress*)

Program rozproszony nazywamy żywotnym, jeśli zapewnia, że każde pożądané zdarzenie w końcu zajdzie, np.: zakończenie algorytmu.

Przykładowo: w odniesieniu do problemu wzajemnego wykluczania własność bezpieczeństwa oznacza, że nigdy dwa procesy nie znajdą się jednocześnie w swoich sekcjach krytycznych, a własność żywotności – że każdy z procesów ewentualnie (w skończonym, ale nieprzewidywalnym czasie) znajdzie się w swojej sekcji krytycznej.

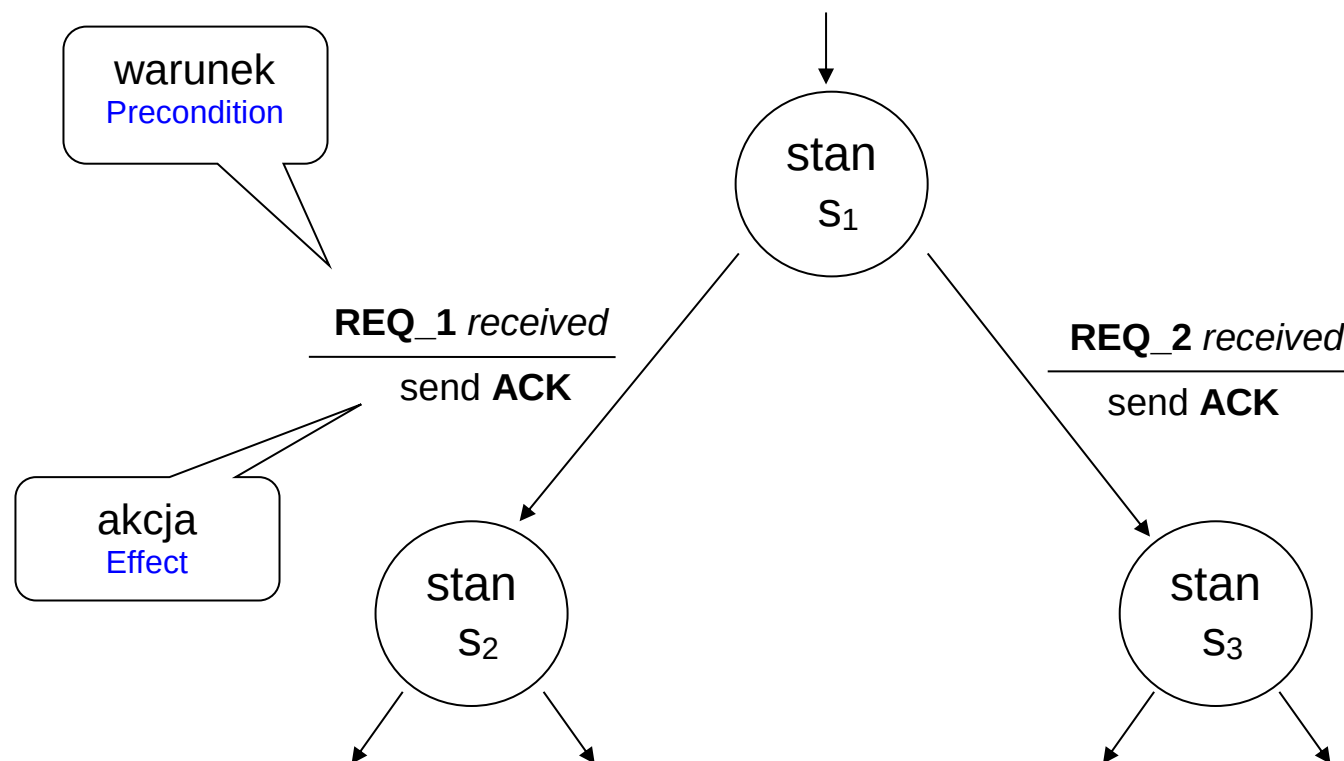
AUTOMAT I/O [4]

Automat I/O $A = \langle \text{Sig}(A), \text{States}(A), \text{Start}(A), \text{Trans}(A), \text{Task}(A) \rangle$

- $\text{Sig}(A)$ – sygnatura (zbiór akcji) automatu:
 $\text{Sig}(A) = \langle \text{Input}(A), \text{Output}(A), \text{Internal}(A) \rangle$
 - $\text{actions}(A) = \text{Input}(A) \cup \text{Output}(A) \cup \text{Internal}(A)$
 - $\text{external}(A) = \text{Input}(A) \cup \text{Output}(A)$
 - $\text{locally_controlled}(A) = \text{Internal}(A) \cup \text{Output}(A)$
- $\text{States}(A)$ – niekoniecznie skończony zbiór stanów
- $\text{Start}(A)$ – niepusty zbiór stanów początkowych
- $\text{Trans}(A)$ – relacja przejść:
 $\text{Trans}(A) \subseteq \text{States}(A) \times \text{Sig}(A) \times \text{States}(A)$
 $\forall_{s \in \text{States}(A)} \forall_{a \in \text{Input}(A)} \exists_{s' \in \text{States}(A)}: (s, a, s') \in \text{Trans}(A)$
- $\text{Task}(A)$ – podzbiory (klasy) akcji automatu kontrolowanych lokalnie ($\text{locally_controlled}(A)$)

akcja a jest
dopuszczalna
w stanie s

Graf stanów automatu





Przykład automatu

Clock

Sig:	IN: request	OUT: return(t), $t \in \mathbb{N}$	INTERNAL: tick
States:	counter $\in \mathbb{N}$, początkowo := 0; flag: Boolean, początkowo := false;		
Trans:	tick	request	return(t)
	Precondition: true	Effect: flag:=true	Precondition: flag==true counter==t
	Effect: counter++		Effect: flag:=false
Task:	{ tick } { return(t): $t \in \mathbb{N}$ }		

Poprawność wykonania automatu A

Wykonanie α automatu A jest **poprawne**

jeżeli dla każdej klasy C operacji z $\text{Task}(A)$:

1. jeśli α jest skończone, to C nie jest dopuszczalne w stanie końcowym wykonania α
2. jeśli α jest nieskończone, to
 - a) α zawiera nieskończenie wiele zdarzeń z C albo
 - b) nieskończenie wiele wystąpień stanów, w których C nie jest dopuszczalne

Przykład dla *Clock*

poprawne wykonania:

tick, tick, tick, ...

tick, tick, request, tick, tick, return(4), tick, tick, ...

niepoprawne wykonania:

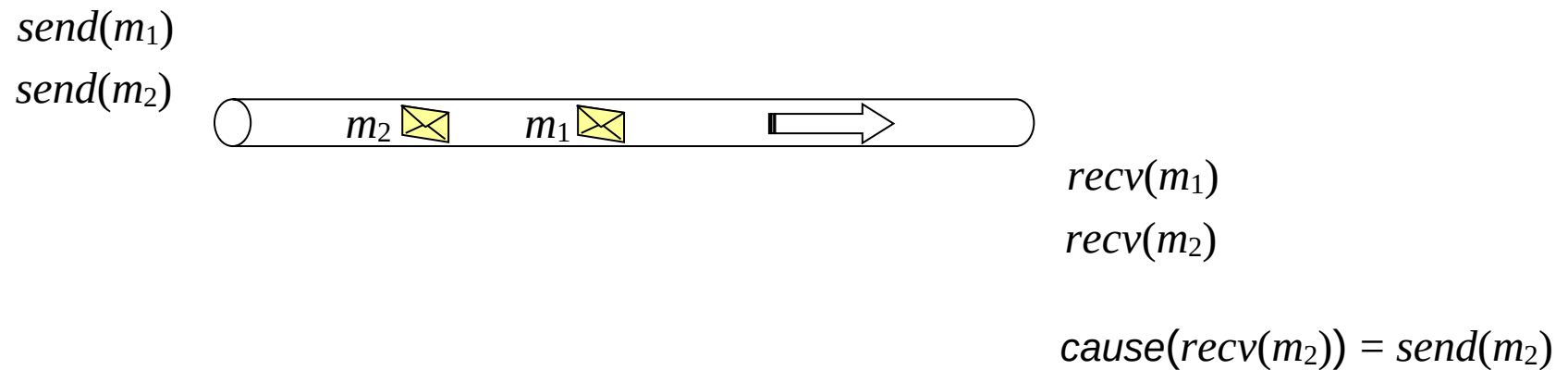
tick, tick, tick

tick, tick, request, tick, tick, tick, ...

KANAŁY KOMUNIKACYJNE

Funkcja *cause*

Funkcja **cause** jest odwzorowaniem każdego zdarzenia *recv* w wykonaniu β na poprzedzające je zdarzenie *send* w tym wykonaniu β :



KANAŁY KOMUNIKACYJNE

Funkcja *cause*

Funkcja **cause** jest odwzorowaniem każdego zdarzenia *recv* w wykonaniu β na poprzedzające je zdarzenie *send* w tym wykonaniu β , takim że:

1. dla każdego zdarzenia $\varepsilon = \text{recv}(m)$, $\text{cause}(\varepsilon) = \text{send}(m)$ z tym samym m
2. *cause* jest odwzorowaniem na zbiór zdarzeń *send*
3. *cause* jest odwzorowaniem 1-do-1 zbiorów zdarzeń *recv* i *send*
4. *cause* zachowuje porządek FIFO

Funkcja *cause* jest narzędziem identyfikacji zdarzenia *send* będącego „przyczyną” wystąpienia zdarzenia *recv* [4]. Warunek 1. wyklucza wygenerowanie wiadomości w kanale komunikacyjnym, warunek 2. wyklucza zagubienie wiadomości, warunek 3. – duplikację, a ostatni warunek narzuca uporządkowanie wiadomości w kanale.

Typy kanałów

niezawodny kanał FIFO (*reliable FIFO channel*)

zachowanie tego kanału opisuje funkcja *cause*

automat *Channel C_{ij}*

Sig:	IN: $send(m)_{ij} \ m \in \mathbb{M}$	OUT: $recv(m)_{ij} \ m \in \mathbb{M}$
States:	Q : kolejka FIFO elementów z $\mathbb{M}$, początkowo $:= \emptyset$;	
Trans:	$send(m)_{ij}$ Effect: $Q := Q \cup \{m\}$	$recv(m)_{ij}$ Precondition: m jest na początku Q Effect: $Q := Q \setminus \{m\}$
Task:	$\{ recv(m)_{ij} \ m \in \mathbb{M} \}$	

kanal niezawodny (*reliable reordering channel*)

zachowuje semantykę exactly-once delivery, bez uporządkowania zdarzeń (funkcja *cause* bez warunku 4.)

Sig:	IN: $send(m)_{ij} \ m \in \mathbb{M}$	OUT: $recv(m)_{ij} \ m \in \mathbb{M}$
States:	$\mathbb{C}$: zbiór elementów z $\mathbb{M}$, początkowo $:= \emptyset$;	
Trans:	$send(m)_{ij}$	$recv(m)_{ij}$
	Effect:	Precondition:
	$\mathbb{C} := \mathbb{C} \cup \{m\}$	$m \in \mathbb{C}$
		Effect:
		$\mathbb{C} := \mathbb{C} \setminus \{m\}$

Oslabiamy własności funkcji *cause*:

Strong loss limitation (SLL)

If there are infinitely many *send*(*m*) events in β execution (for any particular *m*), then there are infinitely many *send*(*m*) events in the range of the *cause* function.

Weak loss limitation (WLL)

If there are infinitely many *send* events in execution β , then the range of the *cause* function is infinite.

Finite duplication (FDp)

The *cause* function maps only finitely many *recv* events to any particular *send* event.

kanał zawodny (*unreliable channel*)

dopuszcza zagubienie i / lub powielenie wiadomości (funkcja *cause* bez warunku 2 i / lub 3)

kanał zawodny (*unreliable reordering channel*) = WLL + FDp + brak uporządkowania

States: $\mathbb{C}$: zbiór z powtórzeniami elementów z $\mathbb{M}$, początkowo $:= \emptyset$;

Trans: $send(m)_{ij}$ Effect: $\mathbb{C} := \mathbb{C} \cup \{m\} \cup \dots \cup \{m\}$ (skończona liczba kopii ≥ 0)	$recv(m)_{ij}$ Precondition: $m \in \mathbb{C}$ Effect: $\mathbb{C} := \mathbb{C} \setminus \{m\}$
--	--

zawodny kanał FIFO (*unreliable FIFO channel*) = WLL + FDp + uporządkowanie

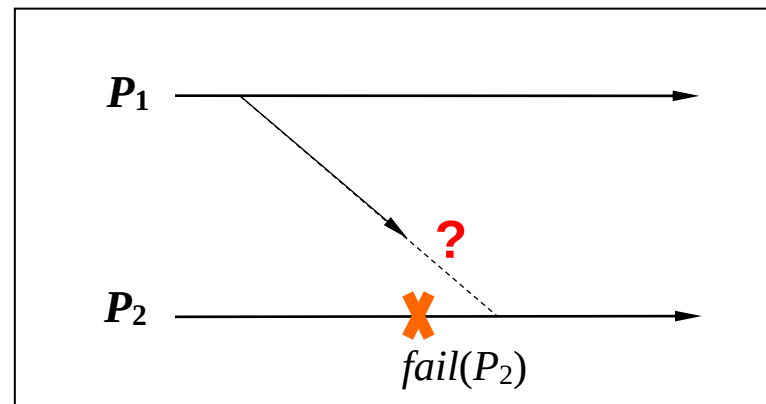
States: $\mathbb{Q}$: kolejka FIFO elementów z $\mathbb{M}$, początkowo $:= \emptyset$;

Trans: $send(m)_{ij}$ Effect: $\mathbb{Q} := \mathbb{Q} \cup [\{m\} \cup \dots \cup \{m\}]$ (skończona liczba kopii ≥ 0)	$recv(m)_{ij}$ Precondition: m jest na początku $\mathbb{Q}$ Effect: $\mathbb{Q} := \mathbb{Q} \setminus \{m\}$
--	--

Awarie koincydentnych procesów

Zachowanie się kanału niezawodnego w przypadku awarii odbiorcy:

- własności kanału niezawodnego wymagają zbuforowania wiadomości w przypadku upadku odbiorcy w trakcie transmisji

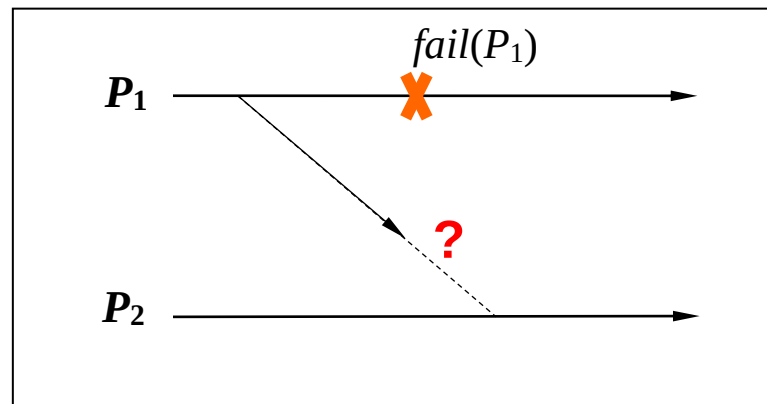


- buforowanie musi odbywać się w podsystemie komunikacyjnym (np. po stronie węzła nadawcy)

Awarie koincydentnych procesów

Zachowanie się kanału niezawodnego w przypadku awarii nadawcy:

- typowo niedostarczenie komunikatu jest dopuszczalne przy upadku nadawcy w trakcie transmisji



- kanały zawsze dostarczające wysłane wiadomości noszą przydomek **uniform** (*uniform reliable channel*)

Awarie koincydentnych procesów

Zachowanie się kanału niezawodnego w przypadku braku awarii:

- kanały dostarczające wiadomości tylko w przypadku, gdy oba procesy (jednocześnie nadawca i odbiorca) są poprawne noszą przydomek **eventual** (*eventual reliable channel*)



ODPORNOŚĆ NA AWARIE (*Failure Resilience*)

Def.: Wykonanie przetwarzania w systemie N procesów nazywamy ***f-odpornym*** (***f-resilient***) jeśli zachowuje poprawność przetwarzania gdy co najmniej $N - f$ procesów jest poprawnych.

Def.: Maksymalna liczba niepoprawnych procesów, przy której dany algorytm (protokół) zachowuje poprawność określa ***stopień odporności na uszkodzenia*** (***fault resilience***) tego algorytmu (protokołu).

NIEZAWODNOŚĆ (*Reliability*)

TOLEROWANIE USZKODZEŃ (*FAULT TOLERANCE*)

maskujące

Def: System zachowuje się zgodnie ze specyfikacją akcji zewnętrznych nawet w trakcie defektu.

bardzo wygodne
i bardzo kosztowne

niemaskujące

Def: Specyfikacja akcji zewnętrznych jest chwilowo naruszona lecz system w skończonym czasie wraca do stanu poprawnego.

idealne, gdy maskowanie jest:

- nieosiągalne (np. consensus)
- niepraktyczne (np. ze wzgl. na koszt)
- niepotrzebne (np. ze wzgl. na wymagania dotyczące systemu)

NIEZAWODNOŚĆ (*Reliability*)

TOLEROWANIE USZKODZEŃ (*FAULT TOLERANCE*)

maskujące

podstawowe techniki:

- redundancja
(np. kodowanie nadmiarowe danych,
replikacja procesów)
- komunikacja grupowa
- uzgadnianie

niemaskujące

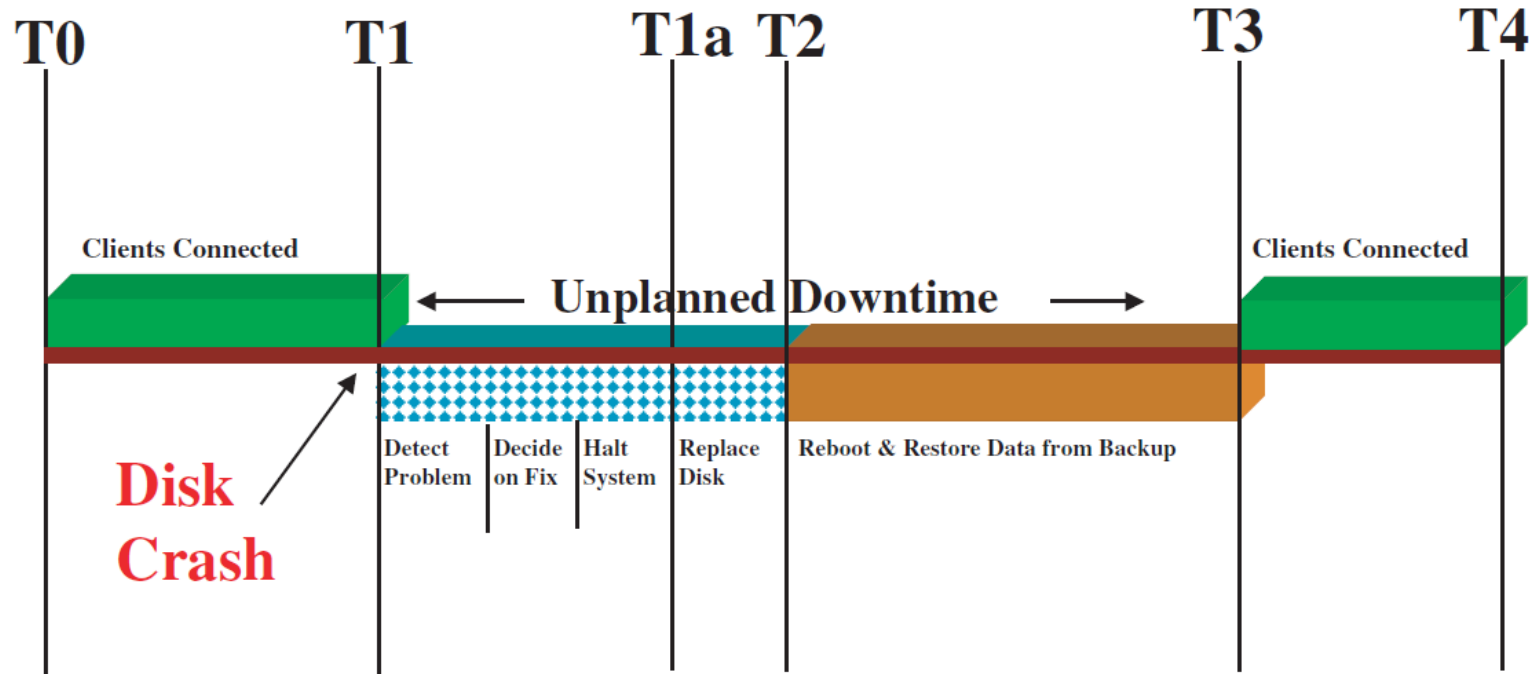
podstawowe techniki:

- odtwarzanie stanu
- stabilizacja

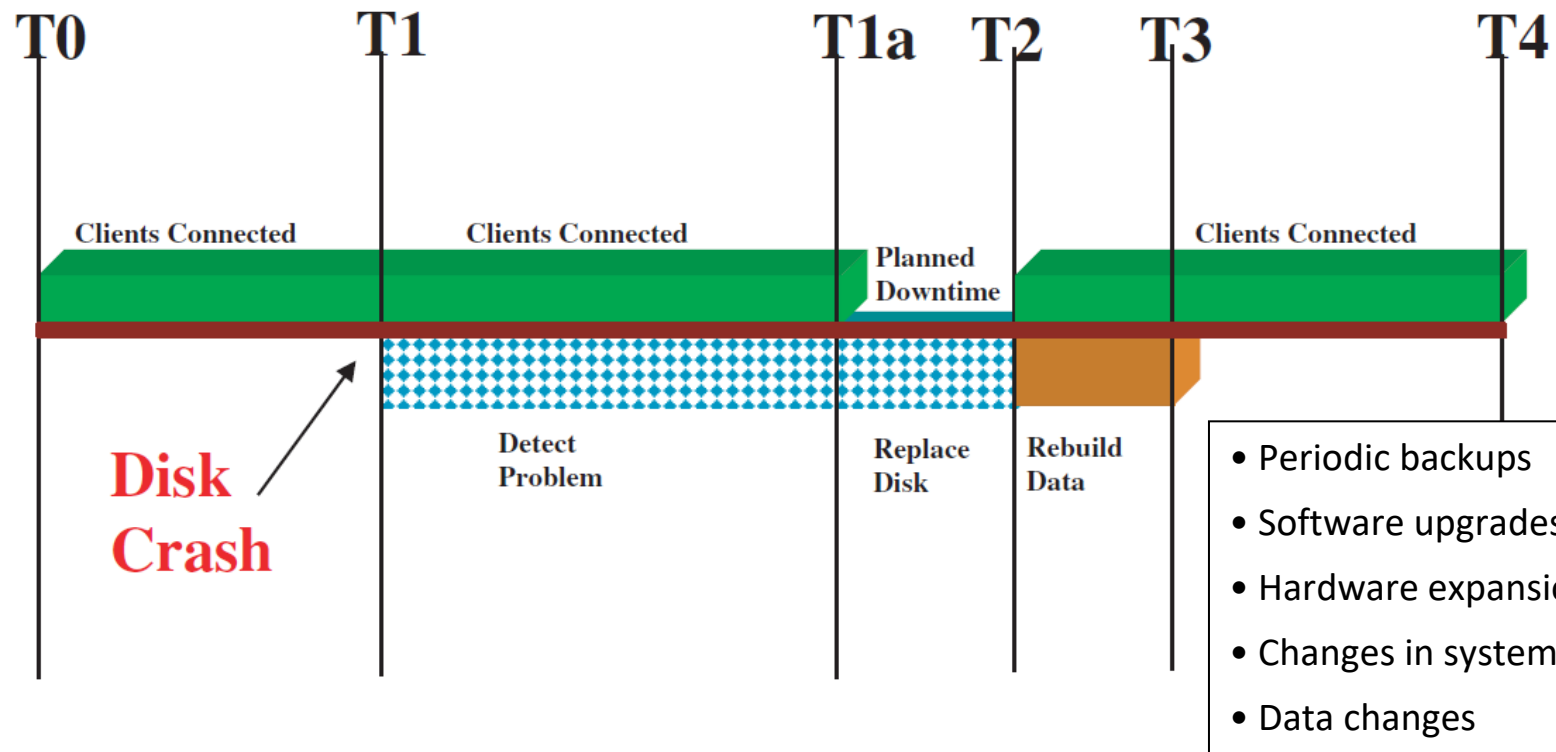
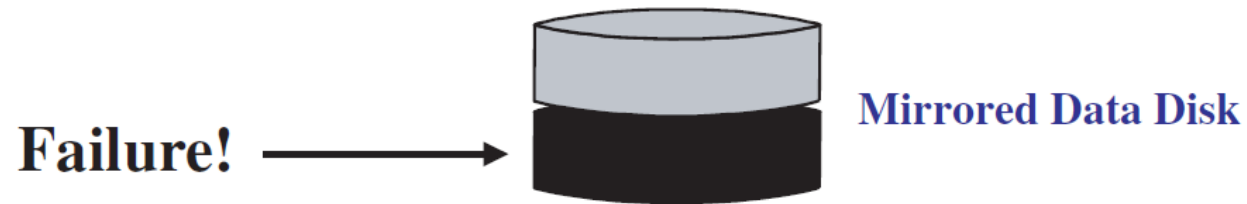
WYSOKA DOSTĘPNOŚĆ (*High availability*)

- ciągła dostępność = 100%
- wysoka dostępność $\neq$ ciągła dostępność
- wysoka dostępność nie ma charakteru absolutnego:
 - wyraża się często jako stosunek czasu ciągłej pracy do całkowitego upływu czasu, np. 99,999%
 - może jej odpowiadać Service Level Agreement (SLA)

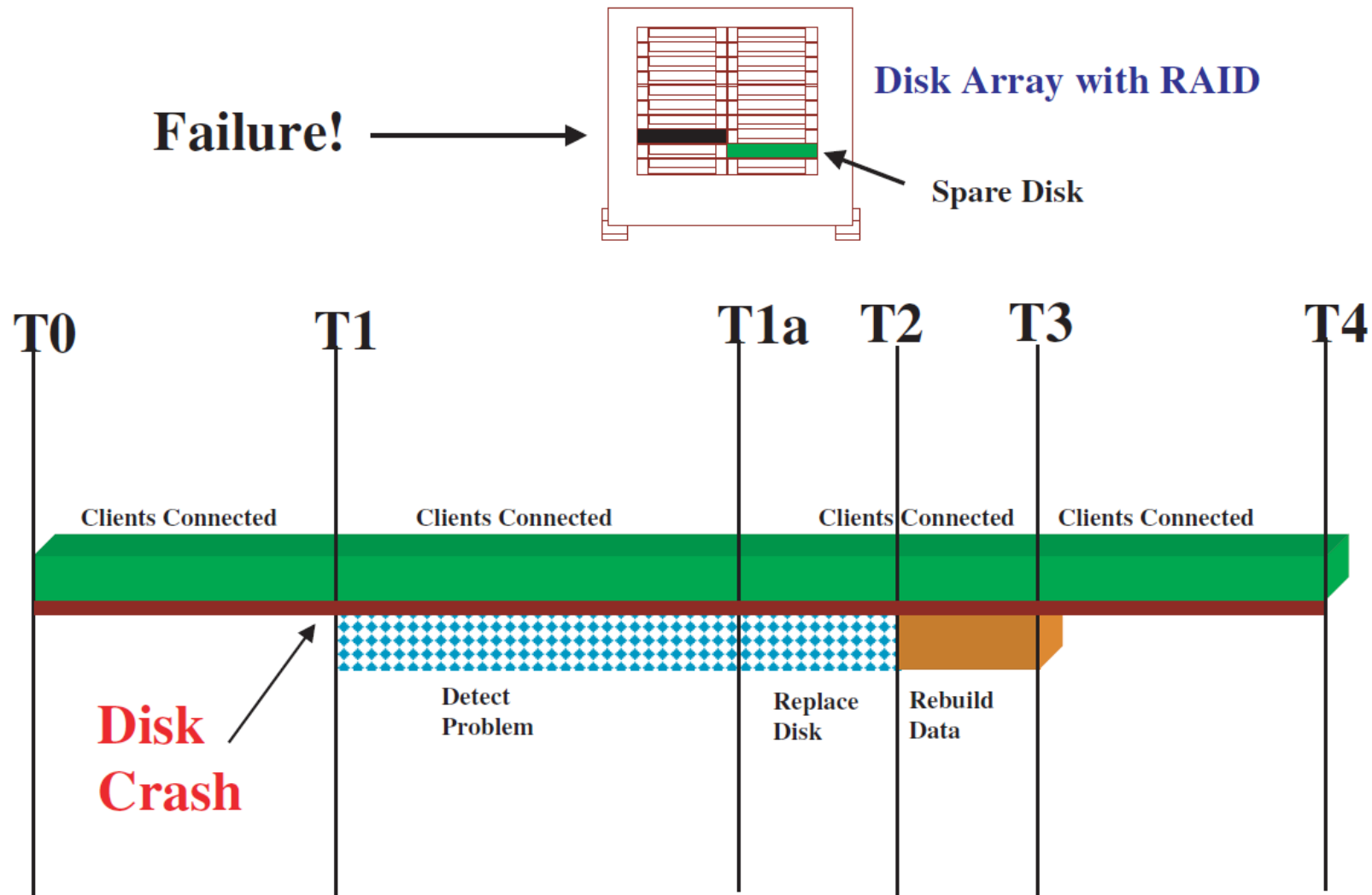
Unplanned downtime [5]



Planned downtime [5]



No downtime [5]



5nines=5minutes

- 99.999% availability = no more than 5 minutes of downtime per year

Percent Available	Seconds Outage per Day	Seconds Outage per Week	Seconds Outage per Month	Seconds Outage per Year
90.000	8,640.000	60,480.000	259,200.000	3,153,600.000
95.000	4,320.000	30,240.000	129,600.000	1,576,800.000
98.000	1,728.000	12,096.000	51,840.000	630,720.000
99.000	864.000	6,048.000	25,920.000	315,360.000
99.500	432.000	3,024.000	12,960.000	157,680.000
99.900	86.400	604.800	2,592.000	31,536.000
99.990	8.640	60.480	259.200	3,153.600
99.999	0.864	6.048	25.920	315.360

- cost grows exponentially with the degree of availability

5nines=5minutes



Forbes Billionaires Innovation Leadership Money Consumer Industry

26,394 views | Aug 19, 2013, 03:50pm

Amazon.com Goes Down, Loses \$66,240 Per Minute

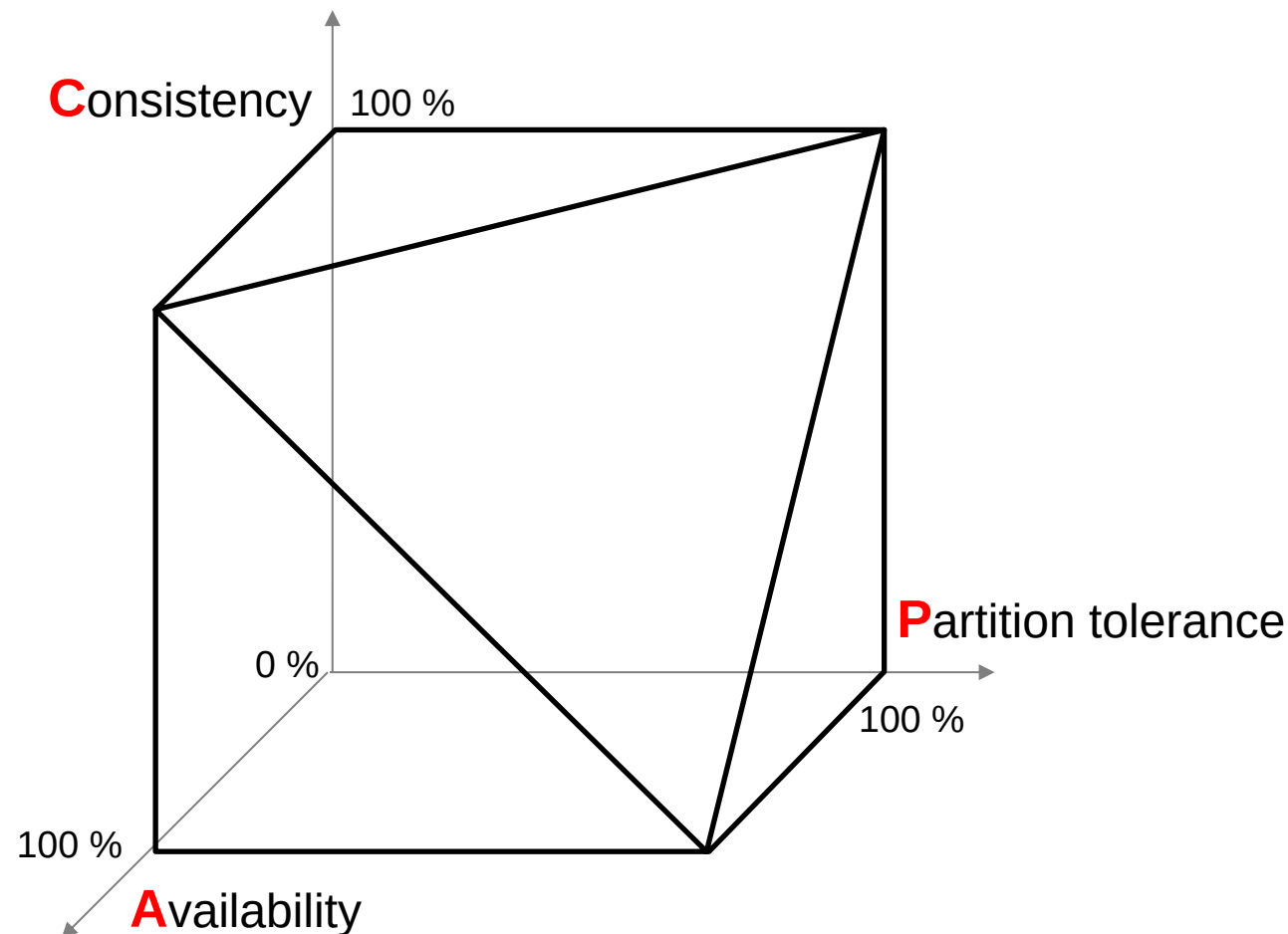
Kelly Clay
Kelly Clay Contributor ⓘ
I write about social media, startups and technology trends.

It's been a bad week for ecommerce. On Friday, Google GOOG -1.92% temporarily went dark, causing a 40% drop in web traffic. Today Amazon.com AMZN -1.96% went down for approximately 30 minutes,

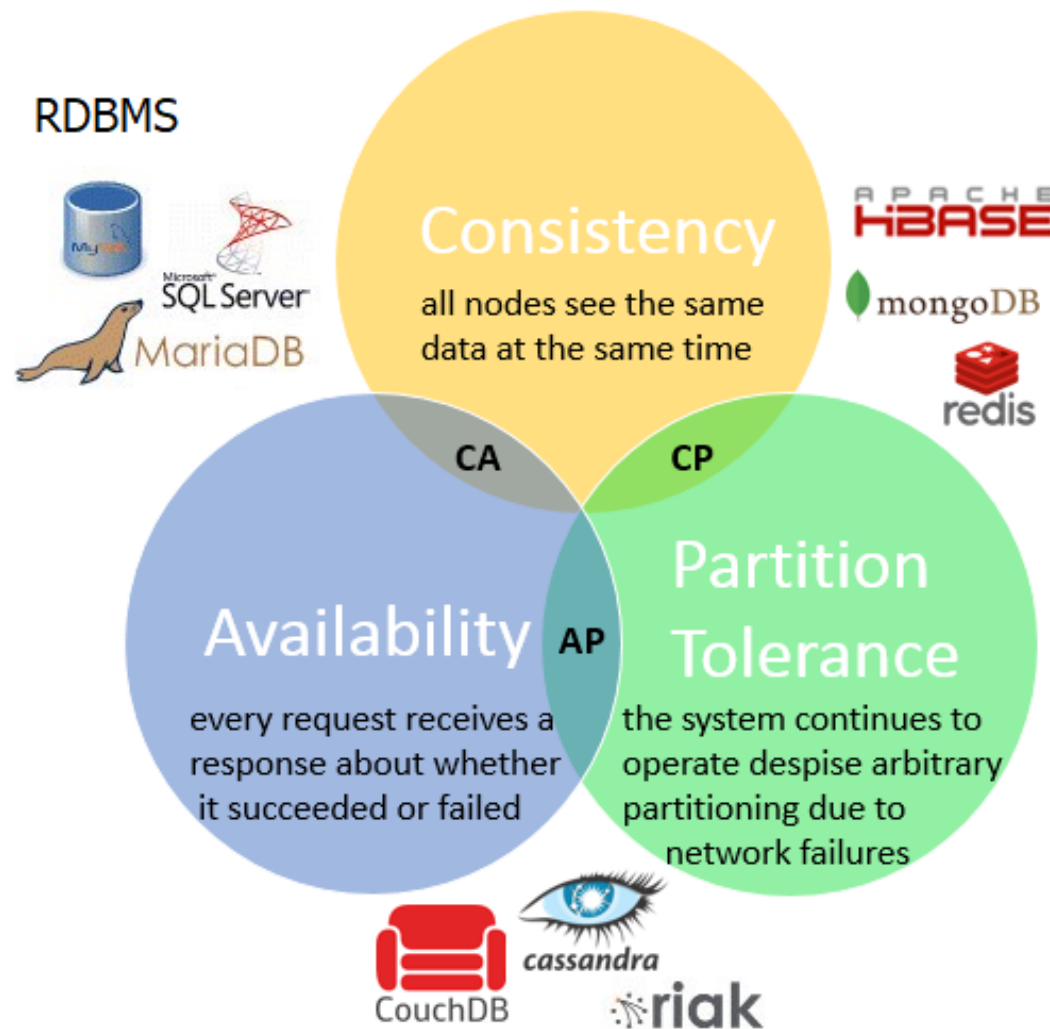


CAP IMPOSSIBILITY RESULT [6]

It is impossible to simultaneously provide all three of the following guarantees:



CAP (CA,CP,AP)



CAP → PACELC

PACELC =

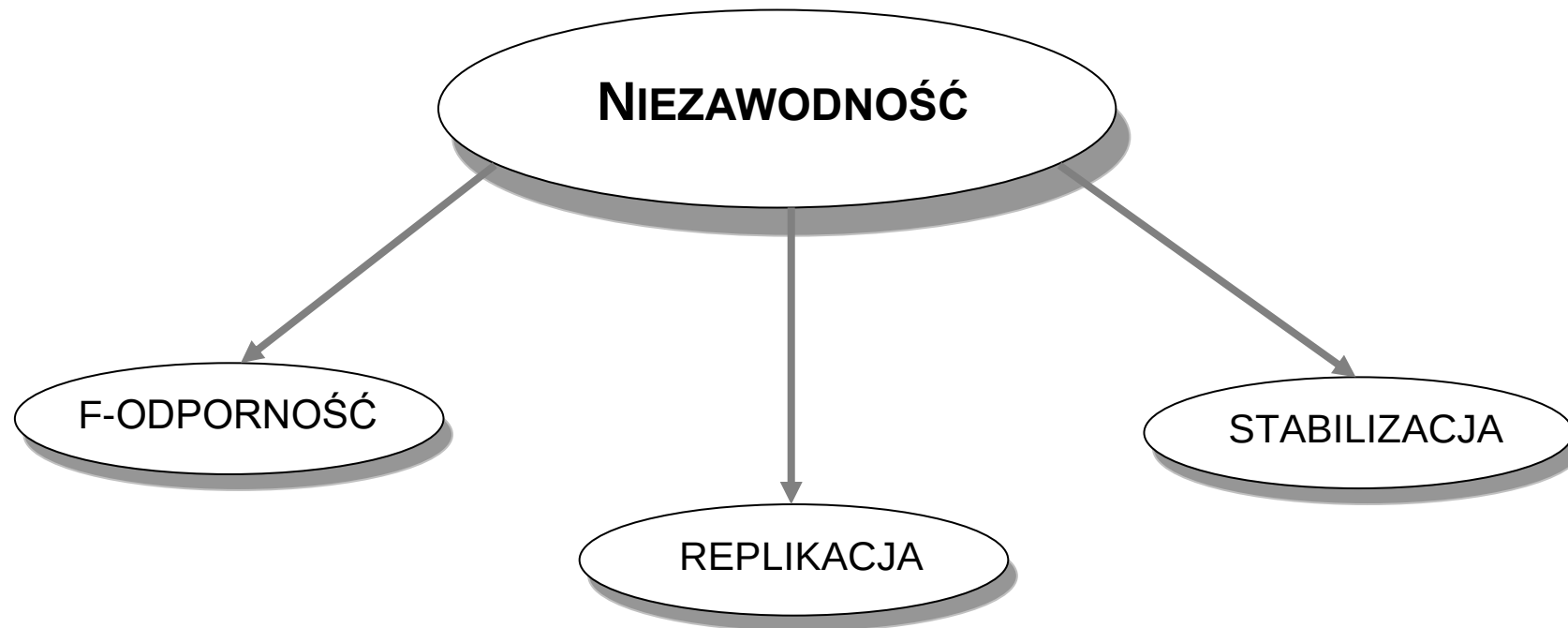
if there is a partition (P):

how does the system trade off availability and consistency (A and C);

else (E), when the system is running normally in the absence of partitions,

how does the system trade off latency (L) and consistency (C)

- the latency/consistency tradeoff (ELC) only applies to systems that replicate data



LITERATURA

- [1]  P.A. Lee, T. Anderson, "Fault Tolerance. Principles and Practice", Dependable Computing and Fault-Tolerant Systems, Springer-Verlag, 1990
- [2]  Jean-Claude Laprie ed., "Dependability: Basic Concepts and Terminology", *Dependable Computing and Fault Tolerant Systems*, vol.5, Springer-Verlag, 1991
tłumaczenie:
Stanisław J. Piestrak, „Wiarygodność: Pojęcia podstawowe i terminologia”, *EDCC-2 Companion Workshop on Dependable Computing*, Politechnika Śląska, Gliwice 1996
- [3]  L. Lamport, "Time, clocks, and the ordering of events in a distributed system", *Communication of the ACM*, vol. 21, no. 7, 1978, pp. 558–565
- [4]  N. A. Lynch, "Distributed Algorithms", Morgan Kaufmann Pub. Inc., 1996
- [5]  Peter S. Weygant, "Clusters for High Availability", Prentice Hall, 2001
- [6]  Eric A. Brewer, "Towards robust distributed systems", Principles of Distributed Computing (PODC 2000), Portland 2000